

```

4 BIT ADDER
module fourbit(a,b,c0,s,c4);
input [3:0]a,b;
input c0;
output [3:0]s;
output c4;
wire c1,c2,c3;
full_adder fa0(a[0],b[0],c0,s[0],c1);
full_adder fa1(a[1],b[1],c0,s[1],c2);
full_adder fa2(a[2],b[2],c0,s[2],c3);
full_adder fa3(a[3],b[3],c0,s[3],c4);
endmodule

module full_adder(a,b,ci,s,c0);
input a,b,ci;
output s,c0;
assign s=a^b^ci;
assign c0=(a&b)|(b&ci)|(ci&a);
endmodule

```

```

4 BIT ADDER TB
module tbfourbit;
reg [3:0]a;
reg [3:0]b;
reg c0;
wire [3:0]s;
wire c4;
fourbit dut(a,b,c0,s,c4);
initial begin
a=4'b0011; b=4'b0111; c0=1'b0;
#10; a=4'b0011; b=4'b0111; c0=1'b1;
#10; a=4'b1101; b=4'b0011; c0=1'b1;
#10;
end initial
#50 $finish;
endmodule

```

```

32 BIT ALU
`timescale 1ns/1ps
module thirtytwobit (
input [31:0] A,
input [31:0] B,
input [3:0] ALU_sel,
output reg [31:0] ALU_out,
output zero
);
assign zero = (ALU_out == 32'b0);
always @(*) begin
case(ALU_sel)
4'b0000: ALU_out = A+B;
4'b0001: ALU_out = A-B;
4'b0010: ALU_out = A&B;
4'b0011: ALU_out = A|B;
4'b0100: ALU_out = A^B;
4'b0101: ALU_out = ~A;
4'b0110: ALU_out = A<<1;
4'b0111: ALU_out = A>>1;
4'b1000: ALU_out = (A<B) ? 32'd1 : 32'd0;
default: ALU_out=32'b0;
endcase
end
endmodule

```

```

TB 32 BIT ALU
`timescale 1ns/1ps
module tbthirtytwobit;
reg [31:0] A;
reg [31:0] B;
reg [3:0] ALU_sel;
wire [31:0] ALU_out;
wire zero;
//Instantiate ALU
thirtytwobit dut(
.A(A),
.B(B),
.ALU_sel(ALU_sel),
.ALU_out(ALU_out),
.zero(zero)
);
initial begin
$display("starting ALU Test...");
A=32'd15; B=32'd10;
ALU_sel=4'b0000;#10;
ALU_sel=4'b0001;#10;
ALU_sel=4'b0010;#10;
ALU_sel=4'b0011;#10;
ALU_sel=4'b0100;#10;
ALU_sel=4'b0101;#10;
ALU_sel=4'b0110;#10;
ALU_sel=4'b0111;#10;
ALU_sel=4'b1000;#10;
$stop;
end endmodule

```

```

4 BIT MOD N COUNTER
module fourbitmodn(
input clk,
input rst_n,
input [3:0] N,
output reg [3:0] count
);
always @(posedge clk or negedge rst_n) begin
if(~rst_n)
count <= 4'b0000;
else if (count==N-1)
count <= 4'b0000;
else
count <= count+1;
end
endmodule

```

```

TB 4 BIT MOD N COUNTER
module tbfourbitmodn;
reg clk;
reg rst_n;
reg [3:0] N;
wire [3:0] count;
fourbitmodn uut(
.clk(clk),
.rst_n(rst_n),
.N(N),
.count(count)
);
always begin
#5 clk = ~clk;
end
initial begin
clk = 0;
rst_n = 0;
N=4;
#10 rst_n=1;
#40;
N=8;
#80;
N=10;
#100;
$finish;
end
initial begin
$monitor("Time=%0t, clk=%b, rst_n=%b, N=%d, count=%d", $time, clk, rst_n, N, count);
end endmodule

```

```

D FF
module dff(D,clk,sync_reset,Q);
input D;
input clk;
input sync_reset;
output reg Q;
always @(posedge clk)
begin
if(sync_reset==1'b1)
Q<=1'b0;
else
Q<=D;
end
endmodule

```

```

TB D FF
module tbdff();
reg D;
reg clk;
reg reset;
wire Q;
dff dut(D,clk,reset,Q);
initial begin
clk=0;
forever #18 clk=~clk;
end
initial begin
reset=0;
D<=0;
#100;
reset=0;
D<=1;
#100;
D<=0;
#100;
D<=1;
$finish;
end
endmodule

```

```

JK FF
module jkff(input reset,input clk,input j,
input k,output reg q,output qnot);
assign qnot=~q;
always @(posedge clk)
if (reset) q<=1'b0;
else
case ({j,k})
2'b00:q<=q;
2'b01:q<=1'b0;
2'b10:q<=1'b1;
2'b11:q<=~q;
endcase
endmodule

```

```

TB JK FF
module tbjkff;
reg clk=0;
reg j=0;
reg k=0;
reg reset=0;
wire q,qnot;
jkff dut (reset,clk,j,k,q,qnot);
initial
begin
#10 j=1'b0;k=1'b0;
#10 j=1'b1;k=1'b0;
#10 j=1'b0;k=1'b1;
#10 j=1'b1;k=1'b1;
#10 j=1'b1;k=1'b1;
#25 reset=1'b1;
#25 $finish;
end
always #1 clk=~clk;
endmodule

```

```

SR FF
module srff(
output reg q,qbar,
input s,r,clk);
always @(posedge clk) begin
if (s==1 && r==0) begin
q<=1;
qbar<=0;
end
else if (s==0 && r==1) begin
q<=0;
qbar<=1;
end
else if (s==0 && r==0) begin
q<=q;
qbar<=qbar;
end
else if (s==1 && r==1) begin
q<=1'bx;
qbar<=1'bx;
end
end
endmodule

```

```

TB SR FF
module tbsrff;
reg s,r,clk;
wire q,qbar;
srff uut (.q(q),.qbar(qbar),.s(s),.r(r),.clk(clk));
initial begin
clk=0;s=0;r=0;
#10 s=1;r=0;
#10 s=0;r=0;
#10 s=0;r=1;
#10 s=1;r=1;
#10 s=0;r=0;
#50 $finish;
end
always #5 clk=~clk;
endmodule

```

```

4 BIT SHIFT ADD
`timescale 1ns/1ps//Define time unit and presicion
module shiftadd (input [3:0] a,input [3:0]
b,output [7:0] product);
reg [7:0] prod_reg;
reg [3:0] count;
integer i;always @* begin
prod_reg=8'b0;
for(i=0;i<4;i=i+1)begin
if(b[i]==1'b1)begin
prod_reg=prod_reg+(a<<i);
end
end
end
assign product=prod_reg;
endmodule

```

```

TB 4 BIT SHIFT ADD
`timescale 1ns/1ps//Define time unit and precision
module tbshiftadd;
reg [3:0] a_tb;
reg [3:0] b_tb;
wire [7:0] product_tb;
shiftadd dut(.a(a_tb),.b(b_tb),.product(product_tb));
initial begin
$display("Time\t a\t b\t | product(Binary) | product(Decimal)");
$display("-----");
//Test cases
a_tb=4'b0000;b_tb=4'b0000;#10;
$display("%0dns:\t %b\t %b\t | %b\t | %0d",
$Time,a_tb,b_tb,product_tb,product_tb);

a_tb=4'b0001;b_tb=4'b0001;#10;
$display("%0dns:\t %b\t %b\t | %b\t | %0d",
$Time,a_tb,b_tb,product_tb,product_tb);

a_tb=4'b0010;b_tb=4'b0011;#10;
$display("%0dns:\t %b\t %b\t | %b\t | %0d",
$Time,a_tb,b_tb,product_tb,product_tb);

a_tb=4'b1010;b_tb=4'b0011;#10;
$display("%0dns:\t %b\t %b\t | %b\t | %0d",
$Time,a_tb,b_tb,product_tb,product_tb);

a_tb=4'b1101;b_tb=4'b0011;#10;
$display("%0dns:\t %b\t %b\t | %b\t | %0d",
$Time,a_tb,b_tb,product_tb,product_tb);
//End simulation
#10;
$stop; end endmodule

```